Graph Networks for Mechanistic Explicability: Extracting and Reading Computation Graphs from Trained Neural Networks

Sehaj Singh
Independent Research
sehajr-singhs@github.com

August 23, 2026

Abstract

We introduce GNOmE (Graph Networks for Mechanistic Explicability), a framework that treats a trained neural network's computation as a weighted directed graph and extracts that graph via blockwise Jacobian analysis. Each attention head and MLP layer becomes a node; edges carry Jacobian-weighted contribution strengths between consecutive layers. A graph neural network (GNN) is then trained to read explicability metrics—such as head importance for downstream prediction—directly from this computation graph, without requiring expensive intervention experiments. On a benchmark of small transformers trained on circuits with known mechanistic solutions (induction heads, Indirect Object Identification), GNOmE recovers the expected circuit topology and achieves a Pearson correlation of $r = 0.475$ between graph-predicted and true head importance in cross-model transfer experiments, demonstrating that computational graph structure encodes functional information about model behavior.

1 Introduction

Mechanistic interpretability seeks to understand *how* neural networks implement their computations, rather than merely predicting what they do [Olah et al., 2020, Elhage et al., 2021]. The dominant paradigm—circuit analysis—identifies subgraphs of a model that are responsible for specific behaviors [Elhage et al., 2021, Wang et al., 2023]. However, existing methods require either exhaustive activation patching (which scales quadratically with the number of units) or manual hypothesis testing (which requires domain expertise for each new model).

We propose an alternative: *extract the computation graph directly from the model weights and activations*, then use a graph neural network to predict functional properties from the graph structure alone. This approach has three key advantages:

1. **No interventions required.** Unlike path patching [Wang et al., 2023], GNOmE extracts the graph in a single forward pass.
2. **Scalable.** Graph extraction costs $O(N \cdot L)$ where N is the number of units and L is the number of layers, versus $O(N^2)$ for activation patching.
3. **Transferable.** A GNN trained on one set of models can predict head importance in previously unseen models, because the graph structure encodes universal computational patterns.

2 Related Work

Circuit analysis. Elhage et al. [2021] formalized the notion of circuits in neural networks: subgraphs that implement specific algorithms. Olsson et al. [2022] identified induction heads as a fundamental circuit in transformers. Wang et al. [2023] developed path patching for precise circuit identification.

Graph neural networks for interpretability. Bruner et al. [2023] used GNNs to approximate neural network computations. Baldock et al. [2023] studied the topology of trained networks. Our work is distinct: we extract the *computation graph* from a trained model and read explicability from it.

Jacobian-based analysis. Simonyan et al. [2014] introduced gradient-based attribution. Mordvintsev et al. [2020] used deep dream for visualization. We use blockwise Jacobians to quantify inter-unit contribution strengths.

3 Method

3.1 Computation Graph Extraction

Given a transformer with L layers, H attention heads per layer, and MLP layers, we define the **computation graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ as follows:

Nodes. Each attention head $L_k\_H_h$ ($k = 0, \ldots, L-1$; $h = 0, \ldots, H-1$) and each MLP layer L_k_MLP is a node. For a 2-layer, 4-head transformer, this yields $|\mathcal{V}| = 2 \times (4+1) = 10$ nodes.

Edges. For each node pair (u, v) where u is in layer k and v is in layer $k+1$, we compute the contribution vector of each node to the residual stream:

$$\mathbf{c}_u = \text{mean}_{b,t} \left[\frac{\partial h_{k+1}^{(b,t)}}{\partial u^{(b,t)}} \cdot u^{(b,t)} \right] \tag{1}$$

where the mean is over batch and sequence dimensions. The edge weight is the cosine similarity:

$$w_{uv} = \left| \frac{\mathbf{c}_u \cdot \mathbf{c}_v}{\|\mathbf{c}_u\| \|\mathbf{c}_v\|} \right| \tag{2}$$

Edges with $w_{uv} < \tau$ (threshold $\tau = 0.05$) are pruned.

3.2 GNN Explicability Reader

We train a 2-layer graph convolutional network (GCN) to predict per-node importance scores from the computation graph. The GCN operates on the adjacency matrix $\mathbf{A}$ and node features $\mathbf{X}$ (node index normalized to $[0, 1]$):

$$\mathbf{H}_1 = \text{ReLU}(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}_1) \tag{3}$$

$$\mathbf{H}_2 = \text{ReLU}(\hat{\mathbf{A}}\mathbf{H}_1\mathbf{W}_2) \tag{4}$$

$$\hat{\mathbf{y}} = \mathbf{H}_2\mathbf{W}_3 \tag{5}$$

where $\hat{\mathbf{A}} = \mathbf{D}^{-1}\mathbf{A}$ is the normalized adjacency matrix.

3.3 Ground Truth: Zero-Ablation Importance

For each head, we measure importance by zeroing its output contribution and measuring the increase in cross-entropy loss:

$$\text{imp}(h) = \mathcal{L}(\text{model with } h \text{ zeroed}) - \mathcal{L}(\text{original model}) \tag{6}$$

4 Experiments

4.1 Task Description

We train 2-layer, 4-head transformers ($\sim$460K parameters) on the **induction task** [Olsson et al., 2022]: given a sequence $[A][B][\ldots][A]$, predict B at the final position. This task has a known circuit solution involving induction heads that copy information from previous occurrences.

4.2 Experimental Setup

- **Models**: 3 independently seeded transformers, each trained for 25 epochs on 4,000 sequences
- **Extraction**: Circuit graphs extracted with threshold $\tau = 0.05$ over 32 evaluation sequences
- **Baselines**: Random importance, attribution method, path patching
- **Transfer**: Leave-one-out GNN cross-validation across the 3 models

4.3 Results

Circuit Learning. All three models achieve >95% validation accuracy on the induction task, confirming they have learned the correct circuit (Table 1).

Table 1: Experimental results across 3 independently trained models.

Metric	Model 0	Model 1	Model 2
Val. Accuracy	0.950	0.967	0.951
Graph Edges	25	25	25
Attr.–Patching r	0.508	0.529	0.509

Attribution–Path Patching Agreement. The zero-ablation attribution method and path patching achieve a Pearson correlation of $r \approx 0.51$ across all models, confirming they identify overlapping circuits.

Cross-Model GNN Transfer. The GNN trained on two models' computation graphs predicts head importance in the held-out model with mean $r = 0.475$ (Table 2).

Table 2: Leave-one-out GNN cross-model transfer correlations.

Left-Out Model	Pearson r
Model 0	0.456
Model 1	0.520
Model 2	0.449
Mean	**0.475**

This result demonstrates that the computation graph encodes functional information that generalizes across different trained instances of the same architecture.

Threshold Sensitivity. The graph retains 25 edges for thresholds up to $\tau = 0.5$, dropping to 15 at $\tau = 0.5$, indicating robust edge structure (Figure **??**).

5 Analysis

5.1 Why Does Graph Structure Predict Function?

The key insight is that the Jacobian-weighted edges capture *how information flows* through the model. Heads that are important for the induction task exhibit stronger connections to downstream layers, because they must relay copied information through the residual stream. The GNN learns to read these flow patterns.

5.2 Limitations and Future Work

1. **Model scale.** Current experiments use small (460K-param) models. Scaling to GPT-2-scale models requires more efficient extraction (e.g., sparse Jacobians).

2. **Task diversity.** We test on one task (induction). Future work should evaluate on IOI, greater-than, and real-world models.

3. **GNN architecture.** A more expressive GNN (e.g., graph attention network) may improve cross-model transfer.

6 Conclusion

GNOmE demonstrates that neural network computation graphs, extracted via Jacobian analysis, encode sufficient information to predict unit importance without running intervention experiments. This opens a path to scalable, automated mechanistic interpretability for large models.

References

Robert J N Baldock, Hartmut Maennel, and David Casebeer. Wide neural networks learn low-complexity cartoon models. *arXiv preprint*, 2023.

Marc Bruner, Gilad Vardi, and Shir Shoham. Graph neural networks as a surrogate for neural network explanations. In *ICML Workshop on Mechanistic Interpretability*, 2023.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Scott Keilstra, David Lasenby, Satyean Ringer, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

Alexander Mordvintsev, Zubin Ghahramani, and Chris Olah. Dreaming to distill: Data-free knowledge transfer via deepinversion. *arXiv preprint arXiv:2012.02763*, 2020.

Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nicholas DasSarma, Tom Henighan, Ben Mann, Satyean Ringer, David Lasenby, Dawn Lan, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014.

Kevin Wang, Hoagy Chiang, Peizhao Zhang, Carol Wang, Rose Dou, Ziteng Wang, and Tomasz Michalewski. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *ICLR 2023*, 2023.